MP4: Scene Notes and Scene Graph

Due Time: Refer to the course web-site

Objective
In this programming assignment, we want to verify our understanding of pivoted transformation, and scene nodes/hierarchy.

Approach
We will build a simple scene manipulation application, one where we can select and manipulate different scene nodes from a scene hierarchy. Here is my attempt at implementation for your reference. For verification, the hierarchy you build must resemble something we, the human, can relate to.

For this assignment, you ***must*** use the 451Shader (where we perform our own model transform) from our lectures, the SceneNode and NodePrimitive classes. There are three main parts to this assignment: 1. Scene Hierarchy, 2. Pivoted Primitive Transform, 3. Simple animations.

Scene Hierarchy: we want to verify two main concepts: A. we can build and manipulate a general hierarchy. B. our hierarchy definition supports reuse. To help verify these, you will:

- **Basic hierarchy:** Create a scene hierarchy with at least four generations of scene nodes (e.g., in my case, Torso, UpperArm, Arm, and Hand), and at least two siblings at one of the generations (e.g., in my case, left and right arms).
  - All scene nodes must be connected to primitives from its previous and subsequent generations.
    - This requirement says, under rotation and scaling, the hierarchy should maintained connected
  - Each scene node must include at least two primitives with one being a sphere located at the joint location, e.g., notice the spheres in between hand/arm, arm/upper-arm, etc.
  - You must support a friendly way for your user to select each of the scene nodes.
  - You must support a friendly way for your user to manipulate the transform of the selected scene node.
  - Your scene hierarchy **behavior must be *intuitive* to a normal human**, in this case, I am the target "normal human". If you are unsure, ask me.
  - The last requirement says, a normal human should be able to look at your hierarchy and have a intuition on what is the correct transform. Putting a collection of random geometries in a hierarchy is not acceptable.
- **Duplicated sub-part:** To verify the definition of your scene hierarchy is reusable, please create a duplication of a sub-part of the hierarchy and parent the sub-part of the hierarchy by the base parent. The duplicated sub-part must contain at least three generations.
  - E.g., my ScaryArm is a simple duplication of RightArm, and has exactly three generations. The pivot position for ScaryArm is moved to the platform.
  - It is ok for the duplicated sub-prat to be identical to the original one.
- **AxisFrame display**: You must support the toggling to display, **prominently**, an axis frame at the pivot position of the currently selected scene node.
  - Only the selected scene node can display the axis frame.
  - Axis frame's orientation must follow the rotation of the scene node.
- **Implementation constraint:** All primitives of your scene hierarchy **must be** shaded by a shader where the model transformation matrix is loaded by you explicitly (e.g., the 451Shader from class examples). You ***cannot*** use any default Unity shaders, access UNITY_MATRIX_MVP, or the _Object2World matrix.
  - Please ask me if you are unsure.
  - A functioning scene hierarchy based on UNITY_MATRIX_MVP or _Object2World matrix will result in a zero credit for this assignment.

Pivoted Primitive Transform: To verify our understanding of the pivoted transform, you will support continuous back-and-forth rotation of:

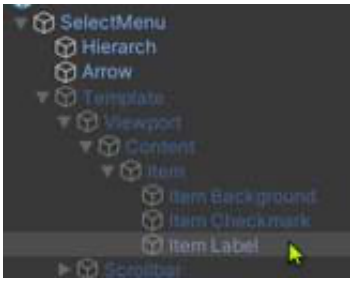
- At least **one sphere**, with rotation around an axis that is tangent of the sphere circumference
- At last **one cube**, with rotation around an axis that is parallel to and located on the surface of the cube

- At least **one capsule**, with pivot located at the long-side tip of the capsule
- The three continuous rotating primitive must
  - Have constant contact with the scene hierarchy (cannot rotate and not touch the hierarchy)
  - Be part of scene nodes from different generations of the scene hierarchy. E.g., in my implementation, I have a sphere/cube/capsule in different level of the scene hierarch. Note that the rotating primitives are always connected to the scene hierarchy, AND, they are part of scene nodes from different generations.
  - You must support at least one rotating primitive at distinctly three different levels.

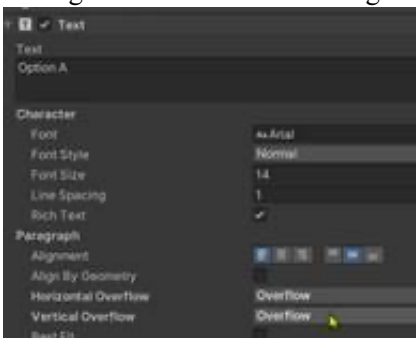
Finally, you must support a **reset** button to reset your scene to the initial position (I did not support this. Sorry).

Hints:

1. You can assume an initial resolution of 1920x1080 (but as previous assignments, please make your EXE Windowed and resizable).
1. Make sure to design your solution without spending extensive time on the UI!
   a. You can spend time experimenting the hierarchy taking advantage of the editor. E.g., when you game runs, observe and manipulate your hierarchy in the Scene window (instead of the Game window).
   b. The number of lines you will need to write for this assignment is really very small, but a little tricky.
   c. **REMEMBER:** editing performed when the game is running WILL NOT BE SAVED!! REMEMBER THIS!
2. Remember to BACKUP often!! I back up by duplicating the entire project folder, about every 20-30 minutes!!
   a. I had to go back to previous saved versions, VERY often.
3. I need to use Awake() // to be called before ANY of the Start(), to make sure that GUI controls are properly initialized
   a. Initialization ordering
      i. GUI initialized is performed first (selection to the base node occurred)
         1. SceneNode system initialized to select the scene node, and thus switching on the AxisFrame for the node
      ii. The scene-node is then called to Start() which assumed it should switch-off AxisFrame.
   b. So, I override Awake() in the SceneNode class to switch-off the display of AxisFrame. Since Awake() is called before GUI's initialization, all is well.
4. Shader. This is the shader I use in my implementation.
5. **XformControl**: You are welcome to use the XformControl from my MP2 solution.
6. **SelectMenu:** if the menu does not show your Node name, it may mean the name is too long and is clamped by the UI display. Two ways to fix this, first, shorten the name, second solution is to disable the label text clamping:
   a. Look at the SelectMenu hierarch, look for Template and ItemLabel …



   b. Change the text overflow settings to ***Overflow*** (bottom of the following screenshot):



Credit Distribution:

Since we don't have control over the MainCamera, your scene MUST clearly display the entire hierarchy.

1.	**Scene Hierarchy:**		**35%**
	a. At least four generations	10%	
	b. Duplication of subpart (e.g. two arms)	10%	
	c. Nodes are connected in generations	15%	
	d. Intuitive select/transform scene nodes	15%	
	e. Proper display AxisFrame of selected	15%	
2.	**Second Hierarchy:**		**25%**
	a. Duplicated subpart from main hierarchy	10%	
	b. At least three generation	10%	
	c. Intuitive select/transform	10%	
3.	**Pivoted Primitive Transform/Animation**		**30%**
	a. Sphere rotation around its circumference	5%	
	b. Cube rotation around its surface	5%	
	c. Capsule rotating about its tip	5%	
	d. All at different generations in the scene hierarchy	5%	
	e. All three always touching the scene hierarchy	15%	
4.	**Scene reset**		**5%**
5.	**Submission:** zipfile content/name, Windowed, Resizable		**5%**

Be aware of the Unity Frustum culling, since we highjacked the vertex shader transform, as far as Unity is concerned, all primitives are located at the origin! If you move the look at position up-wards, the hierarchy will start to disappear! ☺.

Warning: You will receive an automatic zero for the entire assignment if you use any of Unity's shader in your scene hierarchy. Make sure to talk to me if you see the need to use a shader that is very different from what we have developed in class. Once again, there is a relatively small amount of code to be written, BUT, they are tricky. Start early! Let me know your questions so that I can start looking into your problems as soon as you encounter any!

Extra Credit: Some ideas, DO NOT work on the following until you are completely done with the base assignment.

- Duplicate and connect the duplicated hierarchy at the leave (so that your hierarchy is 8 generations deep)
- Define a hierarchy that looks half-way decent (not like mine)
- Include interesting geometries (like a teapot!)